

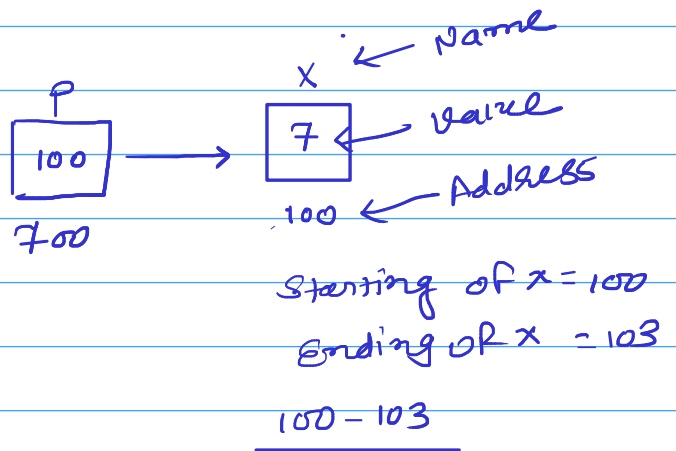
Pointers

* → Value of
& → Address of

int x = 7;

✓ int *P;

✓ float *P2;
✓ char *P3;



① + ②
int *P = &x; ✓

② Assignment
P = 100; ✗
→ P = &x; ✓

0X1A2237

printf("%p", P); // 100
printf("%p", &x); // 100 ✓

③ Accessing the data

P	// 100 → Add. of x	printf("%p", P);	// 100
*P	// 7 → value of x	printf("%d", *P);	// 7
&P	// 700 → Add. of P	printf("%p", &P);	// 700

1) int x=5, y=7, *P, *q;

2) p = &x; ✓

3) y = *P; ✓ // y = x;

4) *P = 70; ✓

5) P = &y; ✓

6) ~~q = &x;~~ ✓

7) ~~q = *P;~~ ✓

8) *q = *P; ✓

9) q = P; ✓

10) *q = 90; ✓

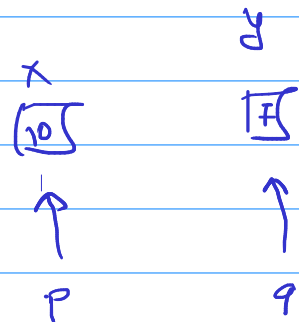
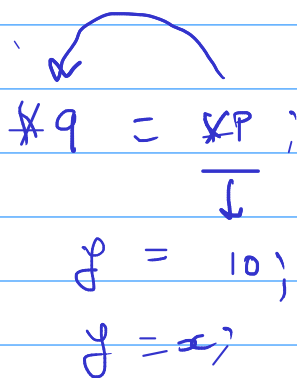
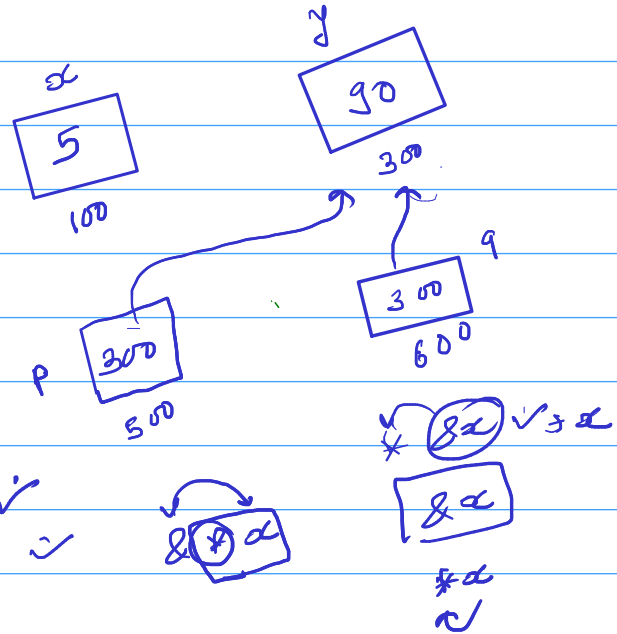
11) *q = 90; ✓

12) *q = 90; ✓

13) printf ("x d x d x d x d", x, y, *P, *q); // 5 90 90 90

14) printf ("x P x P x P x P", &x, &y, &P, &q); // 100 300 500 600

15) printf ("x P x P x P", &y, P, q); // 300 300 300



$\&[*P] \Rightarrow P \Rightarrow \&y;$

$*\&P \Rightarrow P \Rightarrow \&y;$
 $\downarrow$
 $*(350)$
 $\downarrow$
 5000

$\swarrow$
 $\text{int } * \text{ptr};$
 $\text{char } * \text{ptr1};$

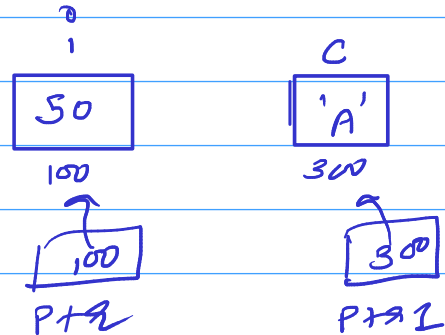
$\text{int } i; \text{ char } c;$

$\text{ptr} = \&i;$

$\text{ptr1} = \&c;$

100

300



$\text{ptr}++;$ ✓

$100 \xrightarrow{\text{ptr}++} 104 \quad (100 + 1 = 104)$

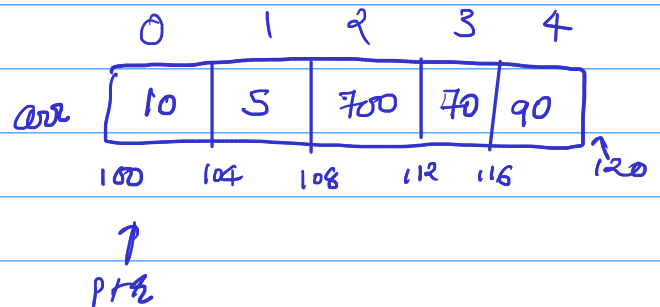
$\text{int arr}[5];$
 $\rightarrow \text{int } * \text{ptr};$

$\text{ptr} = \&\text{arr}[0];$

// $\text{ptr} = \text{arr};$

$*\text{ptr} = 10;$

$\text{ptr} = 10;$ ✗



↑
ptr

↑
// Pointer incrementation

✓ ①

$\text{ptr}++;$

// $100 \rightarrow 104$

Scale factor

✓ ②

$\text{ptr} = \text{arr};$
 $*(\text{ptr} + 3) = 70;$

// Pointer + offset (3)

✓ ③

$\text{printf}(\text{"\%d", * \text{ptr} + 3}, \text{ptr});$ // 100

$\text{arr}[2] = 700;$

// index

$\rightarrow * (\text{arr} + 2) = 700;$ // 100

 $\rightarrow 2 [\text{arr}] = 500;$ valid? ✓

$\rightarrow * (2 + \text{arr})$

→ $ptr[2] = 70;$ ✓
↳ $*(ptr+2)$

→ $2[ptr] = 70;$ ✓

→ $ptr++;$ $*ptr \rightarrow 5$

→ $arr++;$ $*arr \rightarrow 5$ ✗
↑ ↑
Constantly points to base ONLY